Compiler Front-End: Remaining Work and Completion Plan

Status date: 2026-11-03

Scope: Indicode compiler front end only. This plan replaces the previous all-area blueprint PDF.

What “100%” means here

The front end reads source, checks it, reports source-aware errors, and produces a semantically valid, independently verified Indicode HIR. It does not generate LLVM IR, machine code, executables, or run programs.

The current progress report calculates a 95.20% weighted estimate, displayed as 95%. The estimated gap is 4.80 percentage points on that judgment-based scale. This is not a claim that 95% of the language specification or 95% of the PDF catalogue is implemented.

The old PDF described a 777-concept catalogue across the whole language/compiler project. Only 40 catalogue entries (IDs 445–484) are in its compiler-front-end section. In the current implementation audit:

Front-end catalogue status	Count
Implemented to the current scoped acceptance level	9
Partially implemented; more acceptance work remains	17
Not implemented yet	14
Front-end catalogue total	40
Still unfinished (partial + not implemented)	31

These inventory counts and the 95.20% weighted estimate measure different things. The remaining 737 entries in the full catalogue are outside this front-end-only plan; this is not a claim that they are complete.

Remaining catalogue entries

“Done” means the concept has an implementation and focused test for the currently scoped behavior. “Partial” means a subset exists, but the listed completion evidence is still required. “Not started” means there is no implementation to accept yet. Any feature intentionally excluded from the v1 language must be marked as such in the versioned specification and approved before it is removed from this list.

ID	PDF concept	Audit status	Work / evidence still needed
445	Source manager	Done	Source ownership, source IDs, lookup, and bounds tests are implemented.
446	Source buffer	Done	Multiple owned source buffers and stable const access are tested.
447	Source span	Done	Source locations/spans are represented and location boundaries are tested.
448	Line table	Done	CR, LF, and CRLF line boundaries, including trailing empty lines, are tested.
449	Column mapping	Done	Byte offsets, UTF-8 scalar columns, and tab stops are distinct and tested; terminal width policy is tracked under diagnostics.
450	UTF-8 validator	Done	Malformed UTF-8 is diagnosed in source tokens, strings, and comments.
451	Token kind	Partial	Finish the v1 token inventory, including invalid tokens, comments/trivia representation, and every operator/keyword.
452	Token payload	Partial	Define typed literal payloads and preserved source spelling; test payload values and spans for all literal forms.
453	Lexer state	Done	Maintain deterministic EOF/error state and multi-error recovery tests.
454	Trivia preservation	Not started	Preserve comments/whitespace as trivia for formatter/documentation consumers without changing parser behavior.

ID	PDF concept	Audit status	Work / evidence still needed
455	Parser state	Done	Retain explicit parser position/error state and add boundary and malformed-input tests.
456	Recursive descent parser	Partial	Implement the versioned v1 declaration, statement, and type grammar, including imports and generic declarations if accepted in v1.
457	Pratt expression parser	Partial	Specify the complete operator table, associativity, postfix forms, and precedence; add exhaustive precedence tests.
458	Grammar version	Not started	Version the grammar, document compatibility/change policy, and test that docs and parser agree.
459	AST context	Not started	Add a clear AST owner/context and lifetime contract for parsed programs and source references.
460	AST node arena	Not started	Use an arena or document/measure an alternative ownership design; test destruction and large-input behavior.
461	AST visitor	Not started	Provide exhaustive, type-safe visitors and tests that every node kind is visited.
462	AST printer	Partial	Add deterministic AST printer golden tests covering every accepted v1 declaration, statement, and expression node.
463	AST serializer	Done	Keep schema-v1 JSON tests for every node and span; define compatibility policy before treating the schema as stable.
464	Syntax recovery	Partial	Recover at declaration, statement, and expression boundaries; test multiple errors without losing later valid declarations.
465	Error synchronization	Partial	Specify synchronization tokens/sets and add regression tests for cascading-error suppression.
466	Symbol identity	Partial	Add stable identities for functions, types, modules, and imported/overloaded symbols, not only local bindings.
467	Scope table	Partial	Specify namespace separation, visibility, shadowing, and scope lifetime; test nested and module scopes.
468	Module resolver	Not started	Define module identity/search roots/cycles and implement deterministic resolution with positive and negative tests.
469	Import resolver	Not started	Implement import paths, aliases, visibility, ambiguity, and missing-import diagnostics.
470	Overload candidate set	Not started	Define overload selection and ambiguity rules; test exact, failed, and ambiguous candidate resolution.
471	Name binding	Partial	Bind every variable/function/type reference once and diagnose unknown, inaccessible, ambiguous, and invalid references.
472	Type checker	Partial	Complete v1 typing for all declarations/expressions/operators/calls/conversions and reject every invalid combination with stable diagnostics.
473	Constraint solver	Not started	If generics/constraints are in v1, implement constraint collection/solving and inconsistent/unsatisfied-constraint tests.
474	Type substitution engine	Not started	If generics are in v1, implement substitution, occurs/consistency checks as required, and nested-generic tests.
475	Constant evaluator	Partial	Define constant expressions, overflow, evaluation limits, and error behavior; test valid and invalid compile-time expressions.
476	Control-flow validation	Partial	Model reachability and all supported branches/loops; test unreachable/invalid exits and return typing.
477	Definite return analysis	Partial	Prove every reachable path of non-Void functions returns; cover nested branches, loops, and unreachable statements.
478	Effect checker	Not started	Define the v1 effect model (or explicitly defer it by a scope decision) and validate declared/used effects.
479	Ownership checker	Not started	Define ownership, borrowing/lifetimes, and unsafe boundaries (or explicitly defer them); add misuse and valid-flow tests.
480	Tensor shape checker	Partial	Complete static rank/dimension checks and define supported broadcast, dtype, and device compatibility rules; test mismatches.
481	Diagnostic engine	Partial	Add notes/secondary spans, stable severity/code policy, multi-error quality, complete fix-it validation, and golden output tests.

ID	PDF concept	Audit status	Work / evidence still needed
482	JSON diagnostics	Partial	Publish/version the schema and test all fields, locations, notes, fixes, escaping, and compatibility.
483	Language server index	Not started	Build a source-aware symbol/reference index API and test definitions, references, scopes, and invalidation.
484	Incremental parse support	Not started	Define edit ranges/cache invalidation, implement incremental parsing, and prove results match clean full parses.

Work packages and acceptance gates

Implement these in dependency order. A work package is accepted only when its specification, implementation, focused positive/negative tests, and user-facing documentation agree.

A. Freeze the front-end v1 contract

- Document grammar, tokenization, types, integer/float literal rules, conversions, visibility, modules/imports, generics/overloads, tensor checks, ownership/effects, and error behavior.
- For each feature decide: v1 required, explicitly deferred, or out of scope. Do not silently call an undefined feature “complete”.
- Link each normative rule to at least one valid and one invalid test where applicable.

B. Finish source, lexer, parser, and AST infrastructure

- Complete token payloads/trivia and the documented numeric/string/identifier rules.
- Implement the full accepted grammar and precedence/associativity table.
- Complete parser recovery and error synchronization without crashes or loss of later valid declarations.
- Complete AST ownership, visitors, deterministic printing, serialization tests, and schema/version policy.

C. Finish resolution and semantics

- Implement namespaces, visibility, modules/imports, and stable identities across all symbol kinds.
- Implement the accepted overload/generic/type-constraint rules or record a scope decision that defers them.
- Complete constant evaluation, conversions, control-flow/return, ownership/effect, and tensor static checks that remain in v1.
- Keep error codes stable and verify diagnostics at exact source spans.

D. Finish typed HIR

- Lower every accepted v1 construct while preserving resolved identities, types, source spans, and relevant semantic metadata.
- Independently verify all HIR invariants, including symbols, types, control flow, and calls.
- Test AST-to-HIR coverage and JSON round trips/schema compatibility; reject malformed HIR with explicit diagnostics.

E. Finish tooling and release acceptance

- Complete JSON diagnostic schema, fix-it behavior, symbol index, and incremental parse support if retained in v1 scope.
- Add a requirement-linked conformance suite for every v1 grammar/type/diagnostic rule.
- Add coverage-guided lexer/parser fuzzing, sanitizer jobs, clean offline builds, CI, and installed-package consumer checks.
- Ensure all examples and docs match behavior; run all tests from a fresh Release build.

Definition of front-end completion

Report 100% for the agreed v1 front-end scope only after all of the following are true:

1. The v1 contract is versioned and every required feature in the table has an accepted implementation; deferred features are explicitly approved as out of scope.
2. Every requirement maps to passing tests, including malformed input, diagnostics, recovery, and typed HIR invariants.
3. Full conformance, fuzz, sanitizer, clean-build, and package-consumer checks pass in CI.
4. No known required item remains partial or unimplemented; documentation and acceptance evidence are current.

The 16-test suite passing is evidence for its tested cases, not by itself proof of 100% language or production readiness. LLVM, machine-code generation, runtime execution, and AI/device backends remain outside this plan.